DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 Design and Analysis of Algorithms

ASSESSMENT – CO3

Brute-Force and Exhaustive-Search Techniques

Complete Assessment Report

Item	Details
Student name	RITHIKA G
Register number / Section	192571027
Course Outcome	**CO3- Module 3 – Order Lookup (Binary Search)**
Nature / Marks	Individual · 50 marks
GitHub repository	https://github.com/192571027/CSA0607-
Implementation language	Python 3.12 (matplotlib for graphs)

Module 3 – Order Lookup (Binary Search)

Case: Support staff look up an order by timestamp in the consolidated list produced by Module 1. Several orders may share a timestamp and staff need the **first** of them. Lookups currently take seconds.

Chosen algorithm: Binary Search, modified to return the leftmost match (first_occurrence).

---

1. Divide, Conquer, Combine

Step	Binary Search
Divide	Compare the key with the middle element, which splits the sorted array into two halves; $\Theta(1)$.
Conquer	Continue in **one** half only – the other half is provably irrelevant.
Combine	Nothing – the answer of the sub-problem is the answer.

Because only one of the two sub-problems is solved, the recurrence has a single recursive term.

---

2. Why the sorted order from Module 1 is essential

Binary search discards half of the array after one comparison. That is only valid if the comparison tells us *which half can contain the key*: if A[mid] < key, then every element left of mid is also < key – a fact that holds **only if the array is sorted by the search key**.

Demonstration from the program (the unsorted arrival-order list from Module 1's input):

List	Search for 101	Result
Unsorted: (105,O1) (102,O2) (105,O3) (101,O4) (102,O5) (103,O6)	mid = index 2 → 105 > 101 → go left → …	**−1 (not found)** although O4 exists at index 3
Sorted: (101,O4) (102,O2) (102,O5) (103,O6) (105,O1) (105,O3)	finds index 0	correct

So the lookup depends on Module 1 in two ways: the list must be **sorted by timestamp**, and because Module 1 is **stable**, the first order among equal timestamps is also the one that *arrived first* – exactly what support staff expect. If Module 1 were unstable, "first in the list" would be an arbitrary order among the tied ones.

The alternative without sorting is linear search: up to 10,000,000 comparisons per lookup (or an extra index structure to build and maintain).

3. Recurrence and solution

Let T(n) be the number of three-way comparisons (one per iteration), worst case:

$T(1) = 1$

$T(n) = T(\lfloor n/2 \rfloor) + 1 \qquad \text{for } n \geq 2$

Unrolling ($n = 2^k$): $T(n) = T(n/2) + 1 = T(n/4) + 2 = \ldots = T(n/2^k) + k = T(1) + \lg n = \mathbf{\lg n + 1}$.

Master theorem: $a = 1$, $b = 2$, $f(n) = \Theta(1) = \Theta(n^{(\log_2 1)}) = \Theta(n^0) \Rightarrow$ case 2 $\Rightarrow T(n) = \Theta(\log n)$.

General n: $T(n) = \lfloor \lg n \rfloor + 1$ (the number of times n can be halved before reaching 0). Space: O(1) for the iterative version (O(log n) if written recursively).

Case	Comparisons
Best	1 (key at the first midpoint)
Worst / unsuccessful	$\lfloor \lg n \rfloor + 1$

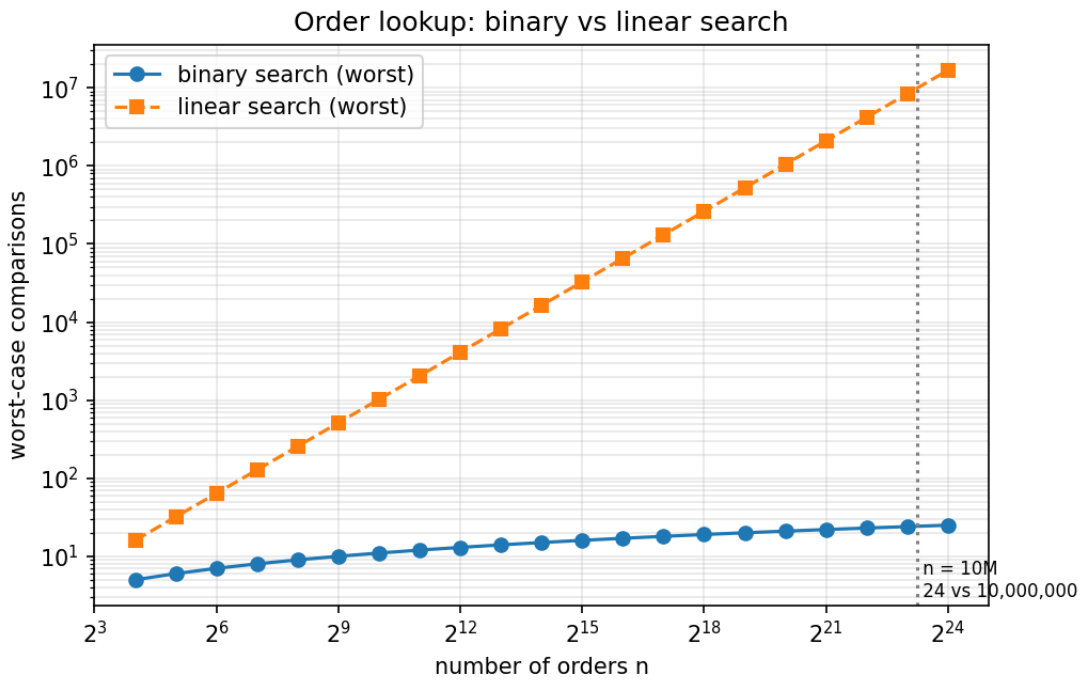
4. Maximum comparisons for 10,000,000 orders

$\lfloor \lg 10{,}000{,}000 \rfloor + 1 = \lfloor 23.2535 \rfloor + 1 = 23 + 1 = 24$ comparisons

Method	Worst case	Average (key present)
Linear search	10,000,000	≈ 5,000,000
Binary search	**24**	≈ 22
Gain (worst case)	10,000,000 / 24 ≈ **416,667** ×	

Verified on a virtual sorted array of 10,000,000 orders (every timestamp repeated three times) with 20,000 random keys plus boundary keys: the **maximum observed was 24** comparisons for the standard search and **25** for the first-occurrence version (explained below).

Note: if each iteration is coded as two two-way tests (== then <), the count doubles to ≤ 48 two-way comparisons – still tiny.



Binary vs linear search

---

5. Returning the first order among equal timestamps

Problem. The standard search stops at the first A[mid] == key it meets. On the sample list, searching 102 returns index 2 (**O5**), although the first order with timestamp 102 is at index 1 (**O2**). Likewise 105 returns index 4 here, but in general the hit can be any of the tied entries.

Modification (lower-bound search). Do **not** stop on equality; instead keep the invariant

every index < lo holds a timestamp < key, and every index ≥ hi holds a timestamp ≥ key.

```
lo ← 0 ; hi ← n
while lo < hi:
   mid ← ⌊(lo + hi)/2⌋
   if A[mid].timestamp < key: lo ← mid + 1      # key cannot be at or left of mid
   else:                      hi ← mid          # mid may itself be the first match
if lo < n and A[lo].timestamp = key: return lo else: NOT_FOUND
```

When the loop ends, lo is the smallest index with timestamp ≥ key; if that timestamp equals the key, it is the first match.

Trace on the sorted sample (from the program)

Search key **102** (A = 101, 102, 102, 103, 105, 105 at indices 0–5):

lo	hi	mid	A[mid]	decision
0	6	3	103	≥ 102 → hi = 3
0	3	1	102	≥ 102 → hi = 1 (do not stop – look further left)
0	1	0	101	< 102 → lo = 1
1	1	–	–	stop; A[1] = 102 ⇒ **index 1, order O2 ✓**

Search key **105**: window shrinks to index 4 ⇒ **O1** (not O3) ✓. Key **104** (absent) and keys outside the range (99, 200) return NOT FOUND.

Key	Standard search	First-occurrence search
102	index 2 → O5 (not the first)	index 1 → **O2**
105	index 4 → O1	index 4 → **O1**
104	−1	−1

Correctness check: 2,000 random sorted arrays with duplicates were compared against a brute-force scan – all matched.

Cost. The loop always runs ⌈lg n⌉ iterations (it cannot stop early) plus one final equality test: $\lceil \lg 10^7 \rceil + 1 = 24 + 1 =$ **25 comparisons** for 10,000,000 orders, compared with 24 for the plain search. One extra comparison buys the correct answer for duplicate timestamps. Space remains O(1).

Extensions. The same idea with ≤ instead of < gives the *last* match; the difference of the two positions counts how many orders share a timestamp, and all of them are contiguous starting at the first index.